CHEAT SHEET · JS INTERVIEW #01 @specsycoder

The Event Loop

Why `setTimeout(fn, 0)` still runs last

index.js

```
1 console.log("A");
2 setTimeout(() => console.log("B"), 0);
3 console.log("C");
```

output A C B

The 4 players

01 Call Stack
Where JS runs code. One stack, one thing at a time.

02 Web APIs
Browser features (timers, fetch, DOM events) that run outside JS.

03 Callback Queue
Finished callbacks wait here in line.

04 Event Loop
Moves a callback to the stack only when the stack is empty.

Step by step

1. `log("A")` goes on the stack, prints A, pops off.
2. `setTimeout` hands its timer to the Web APIs. JS does not wait.
3. `log("C")` runs right away and prints C.
4. The 0ms timer finishes. The callback moves to the queue.
5. Event loop sees the stack is empty, pushes the callback. B prints last.

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

@specsycoder

Promises beat timers

The follow-up question interviewers love

order.js

```
1  console.log("1");
2  setTimeout(() => console.log("2"), 0);
3  Promise.resolve().then(() => console.log("3"));
4  console.log("4");
```

output **1** **4** **3** **2**

FIRST Microtasks

`.then` / `catch` / `finally`, `await`, `queueMicrotask`, MutationObserver

THEN Tasks

`setTimeout`, `setInterval`, UI events, network callbacks, MessageChannel

`sync code → ALL microtasks → one task → ALL microtasks → …`

Traps to mention

- **The delay is a minimum, not a promise.** 0ms means "as soon as possible", never "now".
- **Nested timers get clamped.** After 5 nested levels, browsers enforce at least 4ms.
- **A long loop blocks everything.** Timers and clicks wait until the stack is free.

Your 20-second answer

"JS has one call stack. `setTimeout` hands the callback to a browser timer. When it fires, the callback waits in the task queue, and the event loop runs it only after the stack is empty and all microtasks are done. So 0ms means after the current code, not immediately."